AI HR Recruitment Assistant - Project Documentation

AI HR Recruitment Assistant

Agentic AI HR Recruitment Assistant

Project Documentation

Submitted by

Rajyapriya M

Project Repository
IBM Agentic AI HR Recruitment Assistant

1. Project Overview

Project Title

AI HR Recruitment Assistant: An Agentic AI System for Resume Screening and Candidate Matching

Problem Statement

Recruiters often need to review a large number of resumes for a single job opening. Manual screening is time-consuming, repetitive, and can produce inconsistent results. Important candidate details may also be missed when resumes use different formats or terminology.

This project addresses the problem by providing an AI-assisted recruitment workflow that extracts information from resumes, compares candidates with a job description, ranks candidates, and prepares tailored interview questions.

Brief Description of the Project

The AI HR Recruitment Assistant is a local AI-powered recruitment application built with Python. HR users can enter or upload a job description and upload multiple resumes in PDF, DOCX, or TXT format.

The system extracts resume text, divides it into searchable sections, generates semantic embeddings, and stores the information in a vector database. A locally running Ollama language model analyzes the job description and candidate information. The application then produces candidate match scores, rankings, strengths, weaknesses, recommendations, and interview questions.

The local processing design helps keep sensitive recruitment documents on the user’s machine.

2. Objectives and Proposed Solution

Project Objectives

- Reduce the time required for initial resume screening.
- Compare resumes with job requirements in a consistent way.
- Identify relevant skills and experience from unstructured documents.
- Rank candidates according to their suitability for a role.
- Provide evidence from relevant resume excerpts.
- Generate interview questions tailored to the job and candidate.
- Support local AI processing for improved privacy.
- Provide a simple interface that can be used by HR personnel without programming knowledge.

How the Agentic AI Solution Works

The application follows a multi-stage AI-assisted workflow:

1. **Input collection:** The recruiter provides a job description and uploads one or more resumes.
2. **Document processing:** The system extracts text from PDF, DOCX, and TXT files.

3. **Text chunking:** Long resume text is divided into smaller overlapping sections.

4. **Embedding generation:** Sentence Transformers convert each section into a numerical vector representation.

5. **Knowledge storage:** Resume sections and metadata are stored in ChromaDB.

6. **Job analysis:** Ollama extracts required skills, preferred skills, experience requirements, and education details from the job description.

7. **Retrieval:** Relevant resume sections are retrieved for each candidate using semantic search.

8. **Candidate matching:** The language model evaluates the retrieved information against the job description.

9. **Ranking:** Candidates are sorted by their overall match score.

10. **Interview preparation:** The system generates technical, behavioral, and situational questions for the selected candidate.

The repository also contains an agent workflow layer with separate nodes for document processing, retrieval, matching, ranking, and interview question generation. This structure supports extending the workflow into a more autonomous agent pipeline.

Key Features

- Job description text input.

- Job description file upload.

- Multiple resume uploads.
- PDF, DOCX, and TXT support.
- Resume text extraction and chunking.
- Semantic search using embeddings.
- ChromaDB vector storage.
- Local Ollama model integration.
- Required and preferred skill extraction.
- Candidate scoring and ranking.
- Candidate strengths and weaknesses.
- Strong, Neutral, or Weak recommendation.
- Retrieved resume excerpts for explainability.
- Tailored interview question generation.
- FastAPI generation endpoint.
- Configurable Ollama model, host, and vector storage directory.

3. Implementation and Results

Technologies and Tools Used

Category	Technology	Purpose
Programming language	Python	Application and AI workflow implementation
User interface	Streamlit	Interactive recruiter interface
Backend API	FastAPI and Uvicorn	Text generation API endpoint
Large language model	Ollama	Local job analysis, matching, and question generation
Embeddings	Sentence Transformers	Semantic representation of resume text
Vector database	ChromaDB	Storage and retrieval of resume chunks
PDF processing	PyPDF	Extraction of text from PDF resumes
DOCX processing	python-docx	Extraction of text from Word documents
Numerical processing	NumPy and PyTorch	Embedding and machine learning support
Agent workflow support	LangChain and LangGraph dependencies	Extensible agentic workflow foundation
Version control	Git and GitHub	Project source-code management

Working Process

The main application is started using Streamlit. The recruiter first enters a job description or uploads a job description file. Next, the recruiter uploads one or more candidate resumes and selects **Analyze Candidates**.

For each resume, the application performs the following operations:

- Saves the uploaded file temporarily.
- Extracts its text based on the file format.
- Splits the text into overlapping chunks.
- Generates embeddings for the chunks.
- Stores the chunks and candidate metadata in ChromaDB.

The job description is then analyzed by Ollama. For each candidate, the system retrieves the most relevant resume sections and sends those excerpts with the job description to the matching component. The result is displayed as a ranked candidate list.

The recruiter can select a candidate to view the detailed score, retrieved excerpts, strengths, weaknesses, and recommendation. Interview questions can then be generated using the same job and candidate context.

System Workflow

```
Job Description + Resumes
           |
           v
Document Text Extraction
           |
           v
Text Chunking and Embeddings
           |
           v
ChromaDB Vector Storage
           |
```

```
          v
Job Requirement Analysis
          |
          v
Relevant Resume Retrieval
          |
          v
AI Candidate Matching
          |
          v
Candidate Ranking and Recommendation
          |
          v
Tailored Interview Questions
```

Screenshots and Output

Add screenshots of the following application views to the final report:

1. **Main application screen** showing the job description and resume upload controls.

2. **Candidate ranking screen** showing multiple candidates and their overall match scores.

3. **Candidate details screen** showing the selected candidate’s score, strengths, weaknesses, and recommendation.

4. **Retrieved excerpts screen** showing the resume evidence used for matching.

5. **Interview questions screen** showing generated technical, behavioral, and situational questions.

6. **FastAPI health or generation response** showing the backend API working.

Suggested captions:

- *Figure 1: Job description and resume upload interface.*
- *Figure 2: AI-generated candidate ranking.*
- *Figure 3: Detailed candidate evaluation and retrieved resume evidence.*
- *Figure 4: Tailored interview questions generated for a candidate.*

Results Achieved

The implemented prototype provides an end-to-end recruitment screening workflow. It can:

- Process resumes in common document formats.
- Search resume content based on the meaning of the job description rather than only exact keywords.
- Extract job requirements using a local language model.
- Generate candidate match scores and rankings.
- Display supporting resume excerpts for review.
- Produce interview questions based on the role and candidate profile.
- Operate with a local Ollama model, reducing dependence on external AI APIs.

The system is intended as an AI-assisted screening tool. Final hiring decisions should remain with qualified HR professionals and interviewers.

4. Conclusion and Future Scope

Project Conclusion

The AI HR Recruitment Assistant demonstrates how Agentic AI, retrieval-augmented generation, embeddings, and vector databases can be combined to improve the recruitment process.

By automating repetitive screening tasks, the system helps recruiters review candidates more efficiently while retaining access to the resume excerpts used to support each evaluation. Its local Ollama integration also provides a privacy-conscious approach for handling sensitive recruitment documents.

Challenges Faced

- Resume files have different structures, layouts, and levels of formatting quality.
- Some PDF files may contain scanned images rather than selectable text.
- Language model responses may not always follow the requested JSON format.
- Candidate scoring depends on the quality and completeness of the extracted text.
- Local language models require sufficient system memory and processing resources.
- Vector database compatibility can vary between library versions.

- A reliable production system would require stronger validation, authentication, and audit logging.

Future Enhancements

- Add OCR support for scanned resumes.
- Improve structured extraction using validated Pydantic schemas.
- Add support for more resume formats and multilingual documents.
- Store normalized candidate profiles in a structured database.
- Add configurable scoring weights for skills, experience, and education.
- Provide side-by-side candidate comparison.
- Add recruiter feedback and score correction capabilities.
- Add authentication, role-based access, and encrypted document storage.
- Add audit logs explaining every recommendation.
- Improve duplicate resume detection and candidate history management.
- Deploy the application with Docker and a production database.
- Add automated tests and evaluation datasets for measuring matching quality.

- Add a human approval step before any candidate recommendation is finalized.

References

1. Ollama Documentation: https://ollama.com/
2. Streamlit Documentation: https://docs.streamlit.io/
3. FastAPI Documentation: https://fastapi.tiangolo.com/
4. ChromaDB Documentation: https://docs.trychroma.com/
5. Sentence Transformers Documentation: https://www.sbert.net/
6. PyPDF Documentation: https://pypdf.readthedocs.io/
7. python-docx Documentation: https://python-docx.readthedocs.io/
8. LangChain Documentation: https://python.langchain.com/
9. LangGraph Documentation: https://langchain-ai.github.io/langgraph/
10. Project Repository: https://github.com/rajyapriya05-ctrl/IBM-AGENTIC-AI-HR-RECRUITMENT-ASSISTANT